

Things I've been building, lately

I've been having a lot of fun exploring projects lately. Below are just a handful of a projects I've been messing with. Some are utilities. Some are learning projects. All of them have been incredibly fun and helped along with AI to various degrees.

// beagle

Dynamic, mostly-functional, compiles straight to machine code

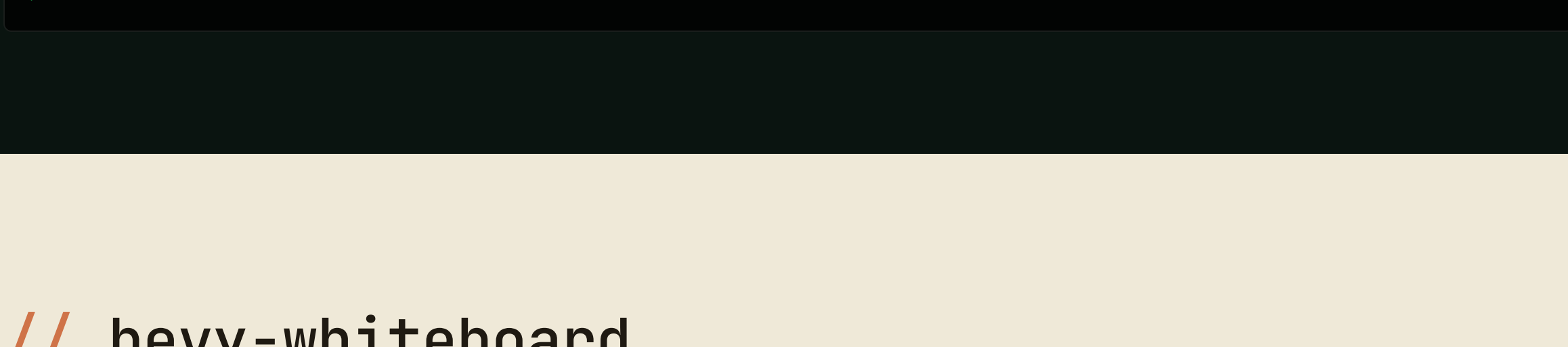
No VM, no bytecode, no JIT warm-up. By far my longest running most complete project. All compilation and code generation are dependency free custom backends for x86 and arm. The goal is to be a language with fantastic live coding support allowing for humans or ai to build full applications without ever restarting a process.



// keep-running

Detach, but with a friendlier CLI and a replay buffer

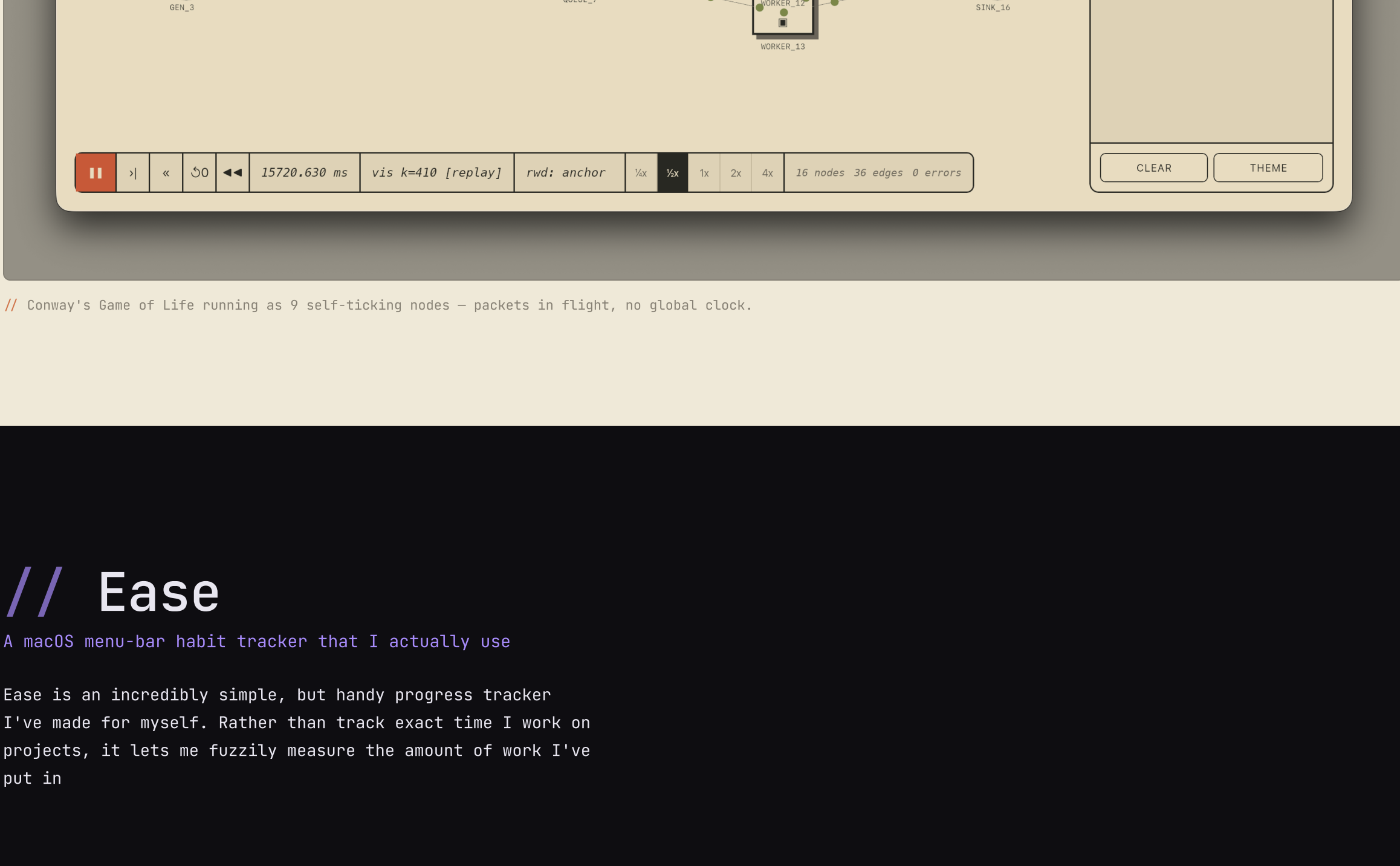
Run a long process, detach, let it keep going, reattach later from anywhere. Small utility, but I've found it incredibly useful



// bevy-whiteboard

The diagram on the whiteboard, but it actually runs

I've always wanted to be able to draw on a whiteboard and then truly simulate the system. This proof-of-concept does exactly that. Under the hood is a simulation driven by a rules-based language. Enabling running simulations faster than real time or rewinding through time. It is quickly approach something powerful and flexible enough to model real systems.



// Conway's Game of Life running as 9 self-ticking nodes – packets in flight, no global clock.

// Ease

A macOS menu-bar habit tracker that I actually use

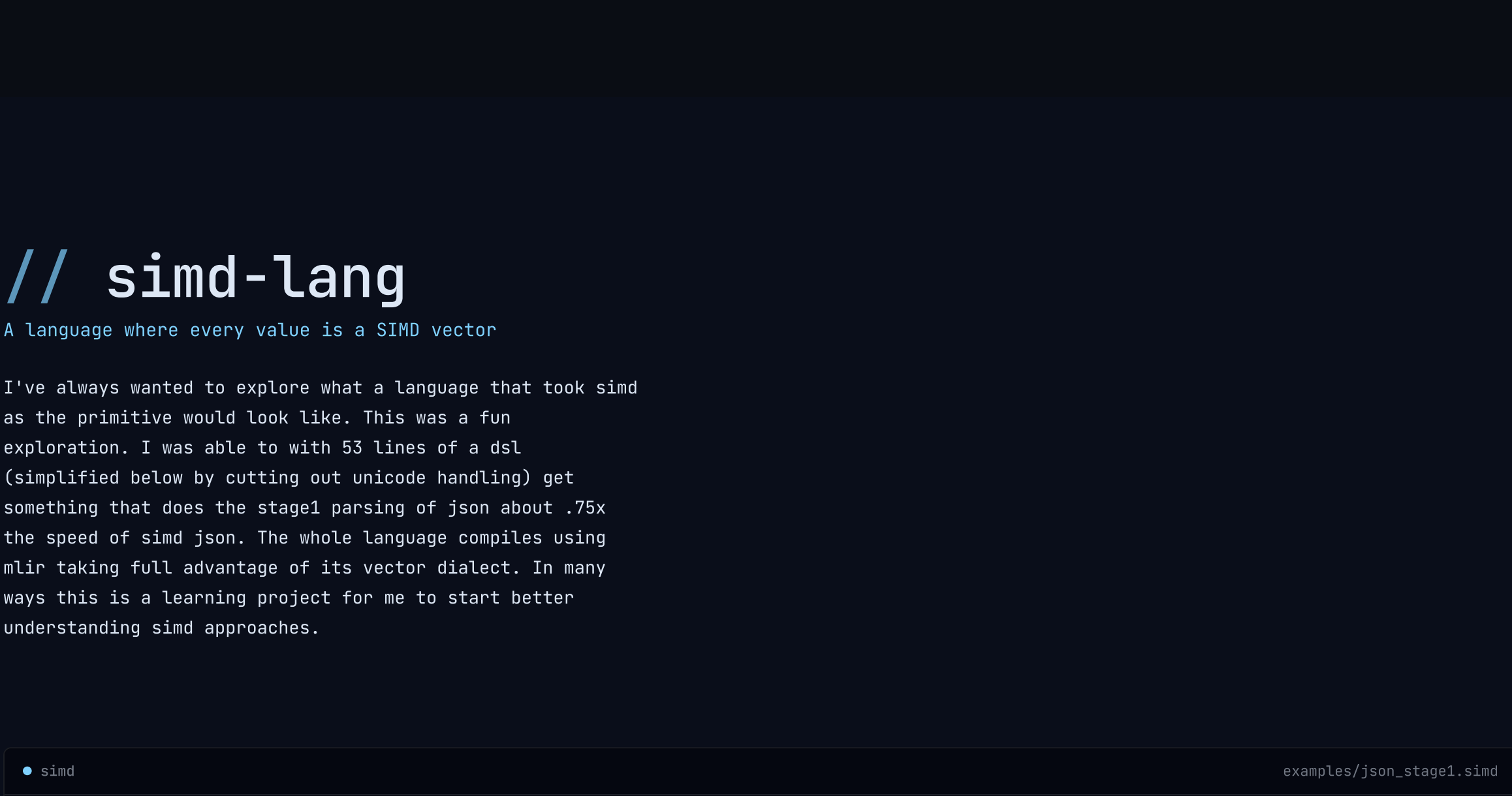
Ease is an incredibly simple, but handy progress tracker I've made for myself. Rather than track exact time I work on projects, it lets me fuzzily measure the amount of work I've put in



// turbopack-visualizer

Untapped Way to Learn a Codebase: Build a Visualizer

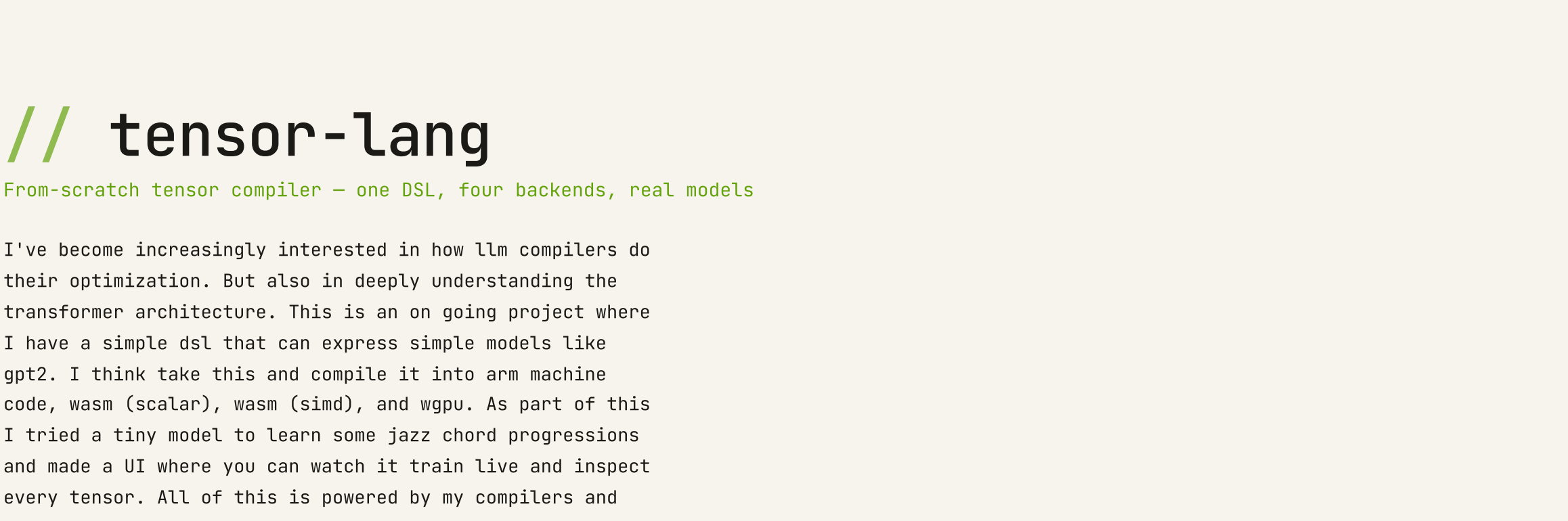
I have aphantasia. From my very first job I realized if I want to fully understand a codebase, I needed to build something that visualizes how it works and let's me interact with it. This is now how I learn new codebases. I wrote a blog post where I walk through how I do this using turbopack as my case study. Below is the video of the quick Vc visualizer and stepping debug that was the end result of the endeavor.



// simd-lang

A language where every value is a SIMD vector

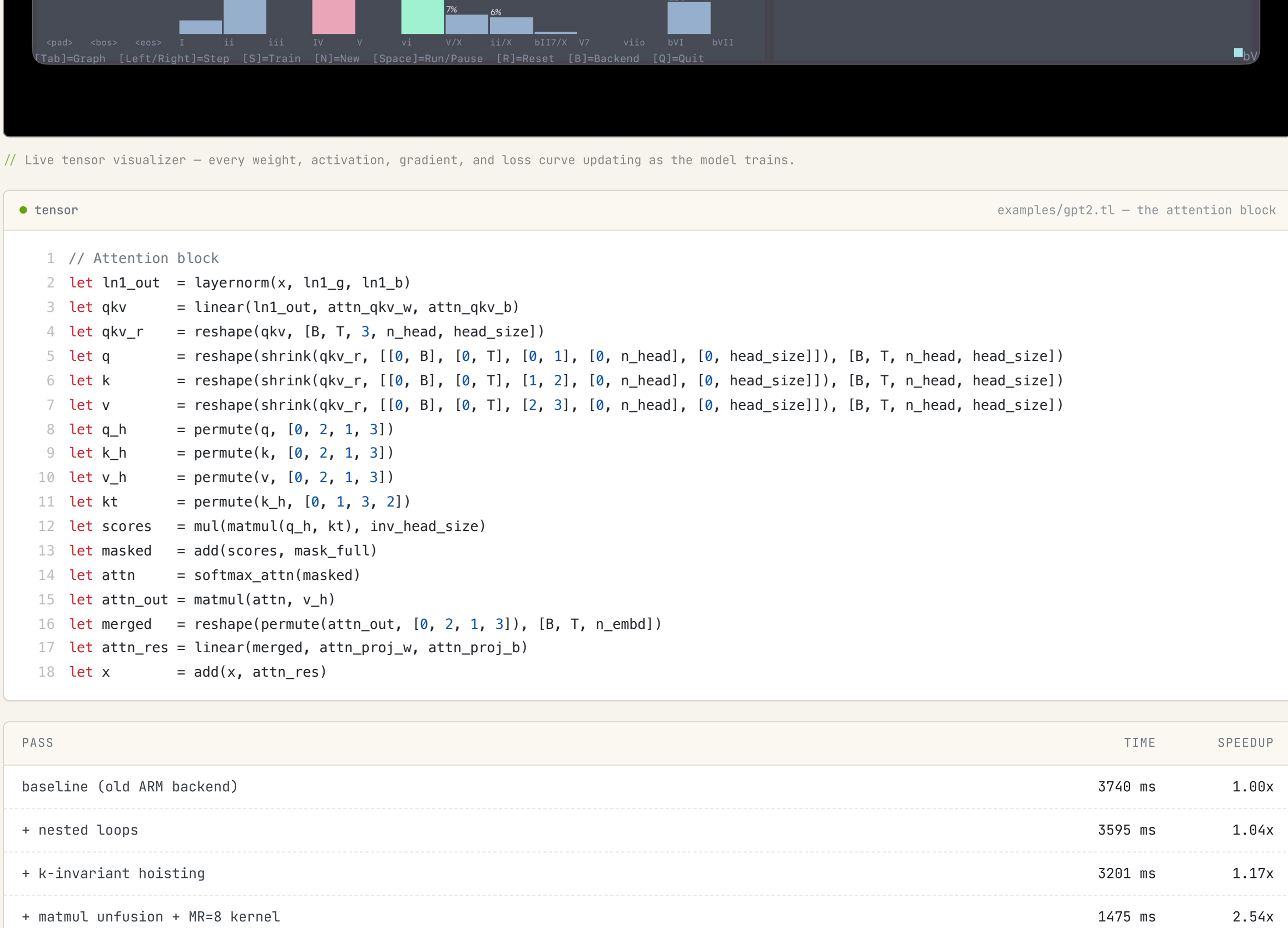
I've always wanted to explore what a language that took simd as the primitive would look like. This was a fun exploration. I was able to with 53 lines of a dsl (simplified below by cutting out unicode handling) get something that does the staged parsing of json about .75x the speed of simd json. The whole language compiles using mllir taking full advantage of its vector dialect. In many ways this is a learning project for me to start better understanding simd approaches.



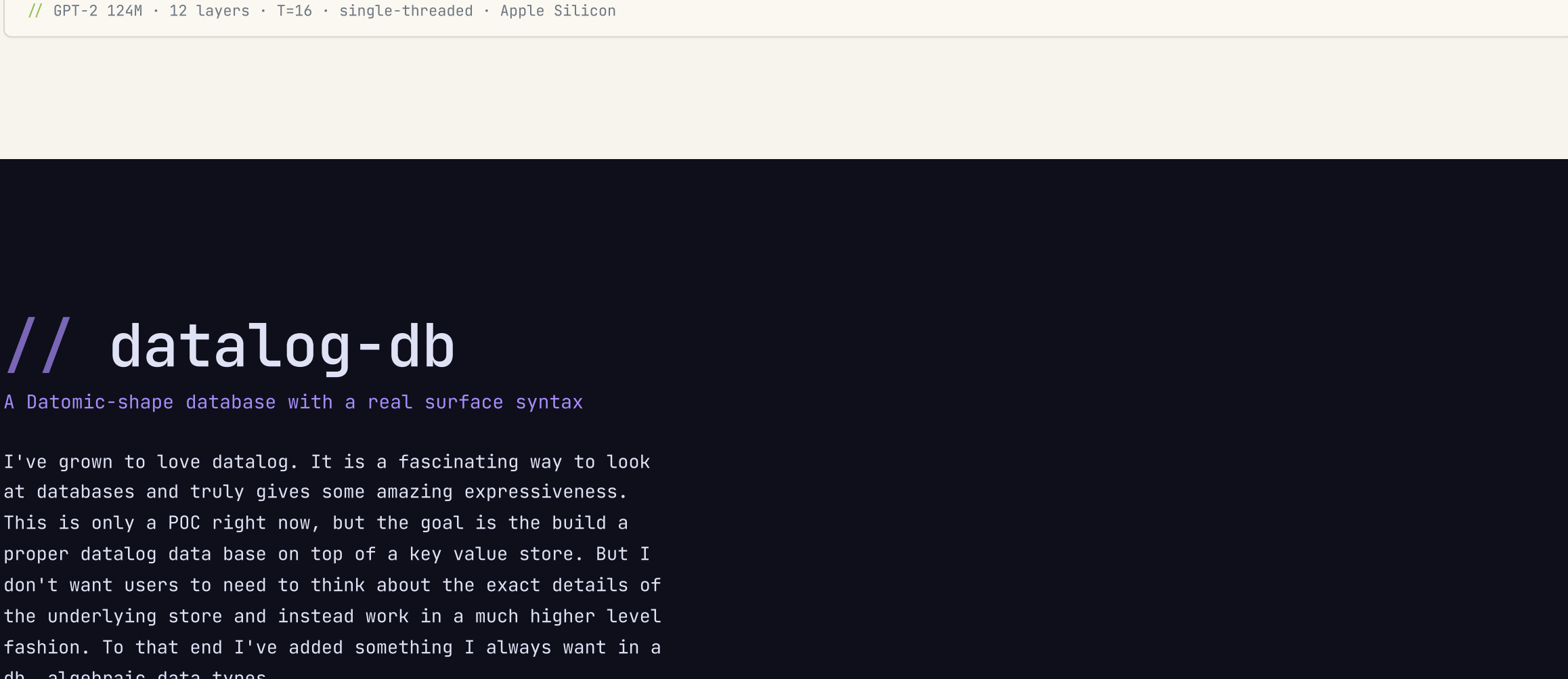
// tensor-lang

From-scratch tensor compiler – one DSL, four backends, real models

I've become increasingly interested in how llm compilers do their optimization. But also in deeply understanding the transformer architecture. This is an on going project where I have a simple dsl that can express simple models like gpt2. I think take this and compile it into arm machine code, wasm (scalar), wasm (simd), and wgpu. As part of this I tried a tiny model to learn some jazz chord progressions and made a UI where you can watch it train live and inspect every tensor. All of this is powered by my compilers and dsl.



// Live tensor visualizer – every weight, activation, gradient, and loss value updating as the model trains.



PASS	TIME	SPEEDUP
baseline (old ARM backend)	3748 ms	1.00x
+ nested loops	3595 ms	1.04x
+ k-invariant hoisting	3201 ms	1.17x
+ matmul unfusion + MR=8 kernel	1475 ms	2.54x
+ pointer incrementing	1434 ms	2.61x
+ all combined	198 ms	18.90x
lln.c -O3 (reference)	84 ms	–
// GPT-2 124M · 12 layers · T=16 · single-threaded · Apple Silicon		

// dataLog-db

A atomic-shape database with a real fascinating syntax

I've grown to love dataLog. It is a fascinating way to look at databases and truly gives some amazing expressiveness. This is only a POC right now, but the goal is the build a proper dataLog data base on top of a key value store. But I don't want users to need to think about the exact details of the underlying store and instead work in a much higher level fashion. To that end I've added something I always want in a db, algebraic data types.

